

Econometrics Softwares

Session 5: Introduction to Machine Learning

Mathilde Col & Jérémy Do Nascimento Miguel
University of Bordeaux

About this course: objectives

- Understand what machine learning does and does not do
 - Lasso is not only useful for cowboys but cannot interpret its coefficients
- Familiarization with recent Machine Learning applications in policy evaluation exercises:
 - In python

1. Introduction to Machine Learning
2. More about Supervised Learning
3. Estimating average treatment effects with many control variables
4. Unpack the treatment effect heterogeneity
5. Other applications: remote sensing or text analysis

Introduction to Machine Learning

Big data?

- Data can be tall (x observations) or **wide** (x covariates)
- With the "Big data revolution" we have more and more data collected (e.g., cell phone data, scanner data) but this is quite different in traditional surveys
- Most of the data we are collecting in development are quite wide $\Rightarrow$ which covariates are relevant for our analysis??

$\Rightarrow$ Machine learning can help to extract and analyze the relevant information

What I mean by Machine Learning?

1. Supervised learning

- If y is continuous: use x to predict y (e.g., y is wheat price, x are farmers, wheat, market characteristics)
- If y is categorical: use x to classify y (e.g., y is cooperative membership, x are farmers characteristics)

2. Unsupervised learning

- Only have x
- Can we predict *unobserved* y clusters?

⇒ Focus on supervised learning: interested in the conditional distribution of y given some covariates x

The basics

Basic terminology

The Machine Learning literature uses a different terminology for the objects of interest

- **Training:** the model is not estimated but trained
- **Training Sample:** the sample used to estimate the function or parameters
- **Testing Sample:** the sample used to predict the outcome
- **Features:** X , regressors, covariates, or predictors
- **Response:** Y , outcome
- **Supervised Learning:** when we observe, and use, both the X and Y
- **Unsupervised Learning:** we only observe the X , and try to group them into clusters
- **Classification Problems:** unordered discrete response problems

Start from the statistical model

$$Y = f(X) + \epsilon$$

with Y =outcome; $X = (X_1, \dots, X_p)$ = predictors; ϵ = mean-zero error independent of X

- This is a *statistical model* meaning that nothing is causal here.
- f represents the *systematic* relationship between x and y
- Objective: estimate f using information X provides about Y , **but** do not care about f itself ($\approx$ *blackbox*)

Why should we estimate $f(X)$

1. Prediction: $\hat{Y} = \hat{f}(X)$; objective is to min. the following function

$$MSE = \frac{1}{n} \sum_{i=1}^N (y_i - \hat{f}(x_i))^2$$

2. Inference:

- Which variables are associated with the outcome
- Relationship between the outcome and each predictor

Assessing model accuracy

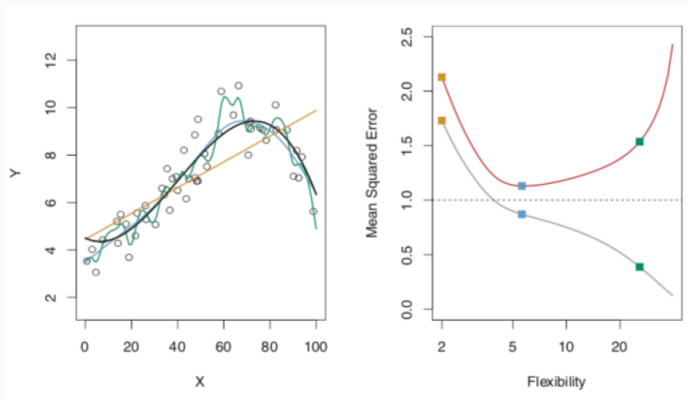
Measure how the predicted variable for an observation is close to the observed one:

$$MSE = \frac{1}{n} \sum_{i=1}^N (y_i - \hat{f}(x_i))^2$$

But this is the **in sample Mean Squared Errors** and used to fit the model. We are interested in the **MSE on test data: out-of-sample-fit**

$$Ave(Y_0 - \hat{f}(X_0))^2$$

Training and Test MSE: an example



From Hastie et al. (2020)

Left: Data simulated from f (black). Three estimates of f : the linear regression line (orange), and two smoothing spline fits (blue and green). Right: Training MSE (grey), test MSE (red). Squares represent the training and test MSEs for the three fits shown in the left-hand panel

Overfitting

Why don't we estimate the most flexible model we can?

- Maximizing flexibility leads to a low out-of-sample fit
- Maximal flexibility leaves all the noise in the prediction model: new observation with the same X has a different idiosyncratic noise, so the prediction is off
- Overfitting arises when a less flexible model would have yield a smaller MSE

Bias-variance trade-off

Overfitting is an example of the bias-variance trade-off. Let's consider the following expected test MSE for a given value x_0 :

$$E[y_0 - \hat{f}(x_0)]^2 = \underbrace{\text{Var}(\hat{f}(x_0))}_{\text{Variance}} + \underbrace{[\text{Bias}(\hat{f}(x_0))]^2}_{\text{Bias}} + \text{Var}(\epsilon)$$

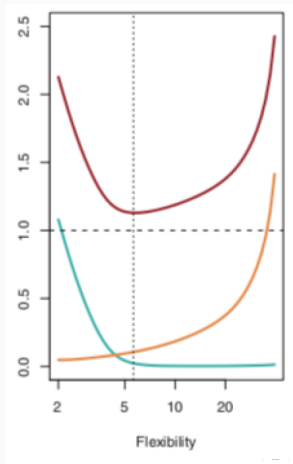
⇒ Need a method such as we have low variance and bias

- Variance: the extent by which $\hat{f}(x)$ would change if we estimated it using a different data set
- Bias: error introduced by approximating a complex general function (i.e., real life problem) by a restricted functional forms. $\hat{f}(X)$ *always* under or over-predicts for some regions of X
- ↑ Flexibility reduces the bias but increases the variance

Practical problems: (i) how do we measure complexity? (ii) Do not observe $E[y_0 - \hat{f}(x_0)]^2$ and need large test set

Bias-variance trade off

$$\text{Test MSE} = \text{bias}^2 + \text{variance} + \text{Var}(\epsilon)$$



- Variance and bias $\downarrow$ as the flexibility increases
- Bias fall rapidly as flexibility increases $\Rightarrow$ sharp decrease in test MSE

$\Rightarrow$ This illustrates the bias-variance trade-off: easy to obtain a method with low variance but high bias

Re-sampling methods

Splitting samples

We are splitting our data set into:

1. Training sample: data used for fitting the model
 2. Testing sample (*hold-out or validation set*): the out-of-sample "sample" from our actual data. Use the model fitted with 1. to predict $\hat{y}$ in this sample
- Resampling means that we are repeatedly drawing sample from 1. to refit the model on each sample to get new information
 - Most used are cross-validation and bootstrap. They allow us to evaluate the model's performance (model assessment) and to select the level of flexibility (model selection)

⇒ Solve our practical problems

Validation-set approach

Easiest way to obtain a testing and training sample: randomly split the dataset into two.

Testing error rate $\approx$ Training error rate

However, most of the time we have access to small data, then a single random split may yield to idiosyncratic differences between the samples/

Drawbacks:

- Testing error rate is **highly variable** depending on which observations
- Use only a subset of the observation to fit the model, while many statistical models perform better with larger sample size $\Rightarrow$ overestimate the test MSE

No cross validation

Take $(x_i, y_i)_{i=1}^n$ and split it into a training and a validation set.



From Hastie et al. (2020)

Pick $c \Rightarrow$ Obtain $\hat{f}^c(x)$ on *training* set $\Rightarrow$ Compute MSE on validation set $\Rightarrow$ Loop over c and choose c that minimizes MSE on validation set

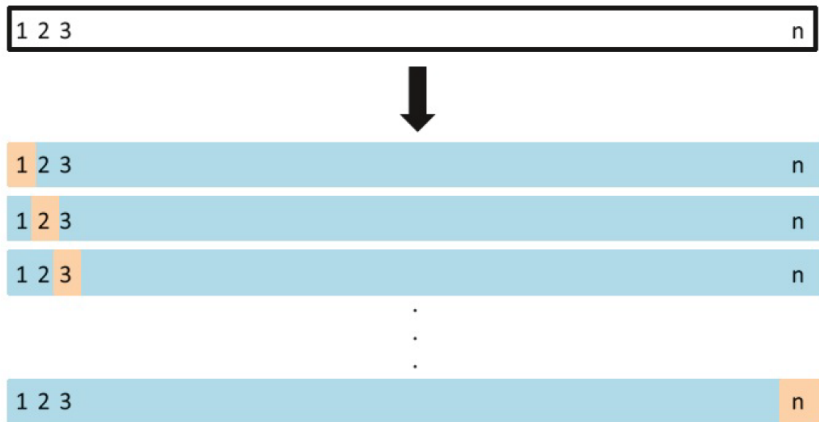
Cross-validation: Leave-one-out cross-validation (LOOCV)

Set of n data points is repeatedly split into two parts: (i) training set with $(n - 1)$ obs.; (ii) testing with 1 obs. leaved out from (i).

- How does it work?
 - Using a model fitted without observation i , $\hat{y}_i$ is the leave-out-one prediction of i
 - Compute $MSE_i = (y_i - \hat{y}_i(x_i))^2$
 - Repeat it for all n observations, leaving one observation out each time
 - Average each test error to obtain the sample test MSE which is:

$$CV_{(n)} = \frac{1}{n} \sum_{i=1}^n MSE_i$$

This is computationally costly because it requires n regressions



From Hastie et al. (2020)

k -fold Cross-Validation

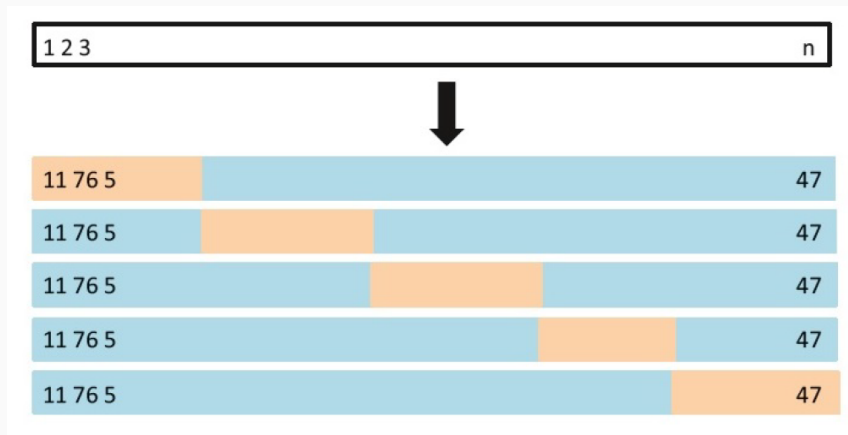
Instead of leaving-out one observation, we randomly **divide our data into k -groups** of $\approx$ similar size

- One fold is the testing sample, the remaining $k - 1$ folds are the training sample
- Fit a model without fold I : $\hat{y}_I$
- Compute $MSE_I = \sum_{j \in I} (y_j - \hat{y}_I(x_j))^2$
- Repeated k times, holding each fold I out successively
- Average each test error to obtain the sample test MSE which is:

$$CV_{(k)} = \frac{1}{n} \sum_{I=1}^k MSE_I$$

This requires only k regressions. Usually $k = 5$ or $k = 10$

5-fold cross-validation



From Hastie et al. (2020)